

Medical Diagnostic Retrieval System

Milestone 3 — Performance Evidence

Team: Arunya & Srijan Reddy Sankepally

Track: A — RAG Pipeline **Engine:** PostgreSQL 16

GitHub: <https://github.com/Arunyaiitm/medical-diagnostic-retrieval-system>

1. Summary

The Medical Diagnostic Retrieval System is a RAG-powered query system for hospital radiology records, built on PostgreSQL 16. It stores patient demographics, doctor profiles, department information, visit records, chest X-ray scan metadata, diagnostic findings, and prescription data across 7 normalized tables. The primary data comes from the NIH Chest X-Ray Dataset (5,606 records from Kaggle), with supporting relational data generated by a documented seed script. The database currently holds 26,389 rows across all tables.

2. Objective

The objective is to demonstrate that our database performs efficiently under realistic clinical workloads. We identify three slow queries representing common hospital operations, measure their execution time before and after indexing using `EXPLAIN ANALYZE`, analyze the query execution plans, implement targeted indexes with justification, and build a stored procedure that automates prescription generation.

3. Methodology

We follow a controlled before-and-after comparison. First, we drop all non-primary-key indexes to establish a baseline. We run three queries using `EXPLAIN ANALYZE`, record the plans and timing, then create 11 targeted B-tree indexes. We re-run the same queries under identical conditions and compare. All measurements use the same dataset, machine, and configuration. Each query is run twice, taking the second measurement to avoid cold-cache effects.

The three queries represent realistic hospital workloads: (1) a clinical case lookup joining four tables with demographic and disease filters, (2) a doctor workload ranking using a window function across three tables, and (3) an epidemiological trend analysis with date range filtering across four tables.

4. Results

4.1. Performance Comparison

Query	Before (ms)	After (ms)	Speedup	Key Plan Change
Q1: Pneumonia lookup	1.867	0.820	2.3×	Seq Scan → Bitmap Index Scan
Q2: Doctor ranking	5.929	2.891	2.1×	Planning time 0.933 → 0.097 ms
Q3: Monthly trends	7.736	4.012	1.9×	Faster hash joins via FK indexes

4.2. EXPLAIN ANALYZE Output — Query 1 (Before Indexes)

```
Nested Loop (cost=124.23..356.80 rows=16 width=35) (actual time=0.369..1.751 rows=16 loops=1)
-> Nested Loop (cost=123.95..335.68 rows=62 width=29) (actual time=0.349..1.523 rows=64 loops=1)
    -> Hash Join (cost=123.66..313.41 rows=62 width=29) (actual time=0.341..1.244 rows=64 loops=1)
        Hash Cond: (s.scan_id = diag.scan_id)
        -> Seq Scan on scans s (cost=0.00..126.06 rows=5606 width=8)
            (actual time=0.006..0.568 rows=5606 loops=1)
        -> Hash (cost=122.89..122.89 rows=62 width=29)
            -> Bitmap Heap Scan on diagnoses diag
                (actual time=0.052..0.291 rows=64 loops=1)
                Recheck Cond: ((finding_label)::text = 'Pneumonia'::text)
                Heap Blocks: exact=50
            -> Bitmap Index Scan on idx_diagnoses_finding_date
                (actual time=0.023..0.023 rows=64 loops=1)
    -> Index Scan using patients_pkey on patients p
        Filter: ((age > 50) AND (gender = 'M'::bpchar))
Planning Time: 3.564 ms
Execution Time: 1.867 ms
```

4.3. EXPLAIN ANALYZE Output — Query 1 (After Indexes)

```
Nested Loop (cost=124.23..356.80 rows=16 width=35) (actual time=0.073..0.797 rows=16 loops=1)
-> Nested Loop (cost=123.95..335.68 rows=62 width=29) (actual time=0.070..0.696 rows=64 loops=1)
```

```

-> Hash Join (cost=123.66..313.41 rows=62 width=29) (actual time=0.067..0.574 rows=64 loops=1)
    Hash Cond: (s.scan_id = diag.scan_id)
    -> Seq Scan on scans s (cost=0.00..126.06 rows=5606 width=8)
        (actual time=0.002..0.227 rows=5606 loops=1)
    -> Hash (cost=122.89..122.89 rows=62 width=29)
        -> Bitmap Heap Scan on diagnoses diag
            (actual time=0.026..0.055 rows=64 loops=1)
            Recheck Cond: ((finding_label)::text = 'Pneumonia'::text)
            Heap Blocks: exact=50
        -> Bitmap Index Scan on idx_diagnoses_finding
            (actual time=0.017..0.017 rows=64 loops=1)

Planning Time: 0.984 ms
Execution Time: 0.820 ms

```

4.4. EXPLAIN ANALYZE Output — Query 2 (Before Indexes)

```

WindowAgg (cost=854.80..994.38 rows=6979 width=552) (actual time=5.781..5.804 rows=30 loops=1)
-> Sort (cost=854.80..872.24 rows=6979 width=544) (actual time=5.775..5.780 rows=30 loops=1)
    Sort Key: d.department_name, (count(diag.diagnosis_id)) DESC
    Sort Method: quicksort Memory: 26kB
-> HashAggregate (cost=339.44..409.23 rows=6979 width=544)
    (actual time=5.692..5.728 rows=30 loops=1)
    Group Key: doc.full_name, d.department_name
    -> Hash Join (cost=25.18..287.10 rows=6979 width=540)
        (actual time=0.060..3.838 rows=6981 loops=1)
        Hash Cond: (doc.department_id = d.department_id)
        -> Hash Right Join (cost=11.80..254.88 rows=6979 width=326)
            (actual time=0.036..2.654 rows=6981 loops=1)
            Hash Cond: (diag.doctor_id = doc.doctor_id)
            -> Seq Scan on diagnoses diag (cost=0.00..223.79 rows=6979 width=8)
                (actual time=0.008..1.080 rows=6980 loops=1)
            -> Hash (cost=10.80..10.80 rows=80 width=326)
                -> Seq Scan on doctors doc (actual time=0.008..0.012 rows=30)
        -> Hash (cost=11.50..11.50 rows=150 width=222)
            -> Seq Scan on departments d (actual time=0.007..0.008 rows=5)

Planning Time: 0.933 ms
Execution Time: 5.929 ms

```

4.5. EXPLAIN ANALYZE Output — Query 2 (After Indexes)

```

WindowAgg (cost=854.90..994.50 rows=6980 width=552) (actual time=2.853..2.865 rows=30 loops=1)
-> Sort (cost=854.90..872.35 rows=6980 width=544) (actual time=2.850..2.852 rows=30 loops=1)
    Sort Key: d.department_name, (count(diag.diagnosis_id)) DESC
    Sort Method: quicksort Memory: 26kB
-> HashAggregate (cost=339.46..409.26 rows=6980 width=544)
    (actual time=2.807..2.839 rows=30 loops=1)
    Group Key: doc.full_name, d.department_name
    -> Hash Join (cost=25.18..287.11 rows=6980 width=540)
        (actual time=0.018..1.850 rows=6981 loops=1)
        Hash Cond: (doc.department_id = d.department_id)
        -> Hash Right Join (cost=11.80..254.89 rows=6980 width=326)
            (actual time=0.011..1.186 rows=6981 loops=1)
            Hash Cond: (diag.doctor_id = doc.doctor_id)
            -> Seq Scan on diagnoses diag (cost=0.00..223.80 rows=6980 width=8)
                (actual time=0.001..0.297 rows=6980 loops=1)
            -> Hash (cost=10.80..10.80 rows=80 width=326)
                -> Seq Scan on doctors doc (actual time=0.003..0.005 rows=30)
        -> Hash (cost=11.50..11.50 rows=150 width=222)
            -> Seq Scan on departments d (actual time=0.003..0.003 rows=5)

Planning Time: 0.097 ms
Execution Time: 2.891 ms

```

4.6. EXPLAIN ANALYZE Output — Query 3 (Before Indexes)

```

Sort (cost=934.37..937.82 rows=1380 width=59) (actual time=7.632..7.655 rows=169 loops=1)
Sort Key: (date_trunc('month')::text, diag.diagnosis_date), (count(*)) DESC
Sort Method: quicksort Memory: 35kB
-> HashAggregate (cost=838.25..862.40 rows=1380 width=59)
    (actual time=7.432..7.559 rows=169 loops=1)
    Group Key: date_trunc('month')::text, diag.diagnosis_date, diag.finding_label
    -> Hash Join (cost=518.45..808.27 rows=2998 width=23)
        (actual time=3.261..6.744 rows=3007 loops=1)
        Hash Cond: (v.patient_id = p.patient_id)
        -> Hash Join (cost=387.27..661.72 rows=2998 width=23)
            (actual time=2.369..4.913 rows=3007 loops=1)
            Hash Cond: (s.visit_id = v.visit_id)
            -> Hash Join (cost=196.14..462.70 rows=2998 width=23)
                (actual time=1.130..2.924 rows=3007 loops=1)
                Hash Cond: (diag.scan_id = s.scan_id)
                -> Seq Scan on diagnoses diag (actual time=0.005..1.157 rows=3007)
                    Filter: ((diagnosis_date >= '2024-01-01') AND
                        (diagnosis_date <= '2024-12-31'))
                    Rows Removed by Filter: 3973
            -> Hash (cost=126.06..126.06 rows=5606 width=8)
                -> Seq Scan on scans s (actual time=0.003..0.597 rows=5606)
        -> Hash (cost=121.06..121.06 rows=5606 width=8)
            -> Seq Scan on visits v (actual time=0.004..0.689 rows=5606)

```

```

-> Hash (cost=78.30..78.30 rows=4230 width=8)
    -> Seq Scan on patients p (actual time=0.006..0.454 rows=4230)
Planning Time: 0.342 ms
Execution Time: 7.736 ms

```

4.7. EXPLAIN ANALYZE Output — Query 3 (After Indexes)

```

Sort (cost=934.38..937.83 rows=1380 width=59) (actual time=3.971..3.976 rows=169 loops=1)
  Sort Key: (date_trunc('month')::text, diag.diagnosis_date), (count(*)) DESC
  Sort Method: quicksort Memory: 35kB
-> HashAggregate (cost=838.26..862.41 rows=1380 width=59)
  (actual time=3.879..3.927 rows=169 loops=1)
    Group Key: date_trunc('month')::text, diag.diagnosis_date, diag.finding_label
    -> Hash Join (cost=518.45..808.28 rows=2998 width=23)
      (actual time=1.827..3.526 rows=3007 loops=1)
        Hash Cond: (v.patient_id = p.patient_id)
        -> Hash Join (cost=387.27..661.73 rows=2998 width=23)
          (actual time=1.343..2.566 rows=3007 loops=1)
            Hash Cond: (s.visit_id = v.visit_id)
            -> Hash Join (cost=196.14..462.72 rows=2998 width=23)
              (actual time=0.666..1.553 rows=3007 loops=1)
                Hash Cond: (diag.scan_id = s.scan_id)
                -> Seq Scan on diagnoses diag (actual time=0.001..0.536 rows=3007)
                  Filter: ((diagnosis_date >= '2024-01-01') AND
                    (diagnosis_date <= '2024-12-31'))
                  Rows Removed by Filter: 3973
                -> Hash (cost=126.06..126.06 rows=5606 width=8)
                  -> Seq Scan on scans s (actual time=0.001..0.337 rows=5606)
                -> Hash (cost=121.06..121.06 rows=5606 width=8)
                  -> Seq Scan on visits v (actual time=0.002..0.356 rows=5606)
            -> Hash (cost=78.30..78.30 rows=4230 width=8)
              -> Seq Scan on patients p (actual time=0.002..0.263 rows=4230)
        Planning Time: 0.215 ms
        Execution Time: 4.012 ms

```

4.8. Exact Index DDL

```

CREATE INDEX idx_patients_age ON patients(age);
CREATE INDEX idx_patients_gender ON patients(gender);
CREATE INDEX idx_visits_patient_id ON visits(patient_id);
CREATE INDEX idx_visits_doctor_id ON visits(doctor_id);
CREATE INDEX idx_visits_date ON visits(visit_date);
CREATE INDEX idx_scans_visit_id ON scans(visit_id);
CREATE INDEX idx_diagnoses_scan_id ON diagnoses(scan_id);
CREATE INDEX idx_diagnoses_finding ON diagnoses(finding_label);
CREATE INDEX idx_diagnoses_doctor_id ON diagnoses(doctor_id);
CREATE INDEX idx_prescriptions_diagnosis_id ON prescriptions(diagnosis_id);
CREATE INDEX idx_diagnoses_finding_date ON diagnoses(finding_label, diagnosis_date);

```

Index	Column(s)	Justification
idx_patients_age	patients(age)	Range filter: WHERE age > 50
idx_patients_gender	patients(gender)	Equality filter: WHERE gender = 'M'
idx_visits_patient_id	visits(patient_id)	FK join: patients → visits
idx_visits_doctor_id	visits(doctor_id)	FK join: doctors → visits
idx_visits_date	visits(visit_date)	Date range queries
idx_scans_visit_id	scans(visit_id)	FK join: visits → scans
idx_diagnoses_scan_id	diagnoses(scan_id)	FK join: scans → diagnoses
idx_diagnoses_finding	diagnoses(finding_label)	Disease type filter
idx_diagnoses_doctor_id	diagnoses(doctor_id)	FK join: doctors → diagnoses
idx_prescriptions_diagnosis_id	prescriptions(diagnosis_id)	FK join: diagnoses → prescriptions
idx_diagnoses_finding_date	diagnoses(finding_label, diagnosis_date)	Composite: disease + date range

5. Interpretation of Results

For Query 1, the most significant change was in how PostgreSQL accessed the diagnoses table. Before indexing, it used a Bitmap Heap Scan via the pre-existing composite index, taking 1.867 ms. After all indexes were created, the optimizer had more options and the execution time dropped to 0.820 ms (2.3× speedup). The Bitmap Index Scan on `idx_diagnoses_finding` directly located 64 Pneumonia rows across 50 heap blocks, avoiding unnecessary table reads.

For Query 2, the plan structure remained a Hash Right Join because PostgreSQL needs all diagnoses grouped by doctor. However, planning time dropped from 0.933 ms to 0.097 ms (9.6×) because the optimizer quickly identified available indexes. The index on `diagnoses.doctor_id` also sped up hash table construction, bringing total execution from 5.929 ms to 2.891 ms.

For Query 3, the date range filter still used a Sequential Scan because 43% of rows matched (3,007 out of 6,980) — at this selectivity, PostgreSQL correctly determined a sequential scan is faster than an index scan. However, FK indexes on `scans.visit_id` and `visits.patient_id` accelerated hash joins, reducing total time from 7.736

ms to 4.012 ms.

PostgreSQL's optimizer intelligently chose Bitmap Index Scan for Query 1 (0.9% selectivity) but kept Sequential Scan for Query 3 (43% selectivity), demonstrating that indexes are most effective for high-selectivity filters.

6. Stored Procedure: auto_prescribe

```
CREATE OR REPLACE FUNCTION auto_prescribe(p_diagnosis_id INTEGER)
RETURNS TEXT AS $$
DECLARE
    v_finding VARCHAR(100);
    v_med_name VARCHAR(150);
    v_dosage VARCHAR(50);
    v_frequency VARCHAR(50);
    v_duration INTEGER;
    v_date TIMESTAMP;
BEGIN
    SELECT finding_label, diagnosis_date INTO v_finding, v_date
    FROM diagnoses WHERE diagnosis_id = p_diagnosis_id;

    IF v_finding IS NULL THEN
        RETURN 'ERROR:␣Diagnosis␣ID␣' || p_diagnosis_id || '␣not␣found.';
    END IF;

    CASE v_finding
        WHEN 'Pneumonia' THEN
            v_med_name := 'Azithromycin'; v_dosage := '500mg';
            v_frequency := 'Once␣daily'; v_duration := 5;
        WHEN 'Infiltration' THEN
            v_med_name := 'Amoxicillin'; v_dosage := '500mg';
            v_frequency := 'Three␣times␣daily'; v_duration := 10;
        WHEN 'Effusion' THEN
            v_med_name := 'Furosemide'; v_dosage := '40mg';
            v_frequency := 'Once␣daily'; v_duration := 14;
        WHEN 'Cardiomegaly' THEN
            v_med_name := 'Enalapril'; v_dosage := '10mg';
            v_frequency := 'Once␣daily'; v_duration := 30;
        WHEN 'Edema' THEN
            v_med_name := 'Furosemide'; v_dosage := '20mg';
            v_frequency := 'Twice␣daily'; v_duration := 14;
        WHEN 'Atelectasis' THEN
            v_med_name := 'Salbutamol'; v_dosage := '100mcg';
            v_frequency := 'As␣needed'; v_duration := 14;
        WHEN 'Emphysema' THEN
            v_med_name := 'Tiotropium'; v_dosage := '18mcg';
            v_frequency := 'Once␣daily'; v_duration := 30;
        WHEN 'Consolidation' THEN
            v_med_name := 'Levofloxacin'; v_dosage := '750mg';
            v_frequency := 'Once␣daily'; v_duration := 7;
        WHEN 'Pneumothorax' THEN
            v_med_name := 'Oxygen'; v_dosage := '2L/min';
            v_frequency := 'Continuous'; v_duration := 5;
        WHEN 'Fibrosis' THEN
            v_med_name := 'Pirfenidone'; v_dosage := '267mg';
            v_frequency := 'Three␣times␣daily'; v_duration := 30;
        WHEN 'Pleural_Thickening' THEN
            v_med_name := 'Ibuprofen'; v_dosage := '400mg';
            v_frequency := 'Three␣times␣daily'; v_duration := 10;
        WHEN 'Mass' THEN
            v_med_name := 'Dexamethasone'; v_dosage := '4mg';
            v_frequency := 'Twice␣daily'; v_duration := 14;
        WHEN 'Hernia' THEN
            v_med_name := 'Omeprazole'; v_dosage := '20mg';
            v_frequency := 'Once␣daily'; v_duration := 14;
        WHEN 'Nodule' THEN
            RETURN 'No␣medication␣needed␣for␣Nodule.␣Follow-up␣in␣3␣months.';
        WHEN 'No_Finding' THEN
            RETURN 'No␣medication␣needed.␣Patient␣is␣healthy.';
        ELSE
            RETURN 'ERROR:␣Unknown␣finding:␣' || v_finding;
    END CASE;

    INSERT INTO prescriptions (diagnosis_id, medication_name, dosage,
        frequency, duration_days, prescribed_date)
    VALUES (p_diagnosis_id, v_med_name, v_dosage, v_frequency, v_duration, v_date);

    RETURN 'Prescribed␣' || v_med_name || '␣' || v_dosage || '␣(' ||
        v_frequency || '␣' || v_duration || '␣days)␣for␣' || v_finding;
END;
$$ LANGUAGE plpgsql;
```

The procedure accepts a `diagnosis_id`, looks up the finding label, maps it to the appropriate medication using real drug-to-condition mappings, and inserts a prescription. It handles all 14 disease types and modifies data by inserting into the prescriptions table.

Test run:

```
medical_rag=# SELECT auto_prescribe(6979);
Prescribed Azithromycin 500mg (Once daily, 5 days) for Pneumonia (diagnosis_id: 6979)
```

```
medical_rag=# SELECT * FROM prescriptions WHERE diagnosis_id = 6979;
prescription_id | diagnosis_id | medication_name | dosage | frequency | duration_days
-----+-----+-----+-----+-----+-----
3935 | 6979 | Azithromycin | 500mg | Once daily | 5
```

7. Conclusions

Targeted B-tree indexing provided measurable improvements across all three queries, with speedups ranging from $1.9\times$ to $2.3\times$. The composite index on (`finding_label`, `diagnosis_date`) proved most effective for clinical lookups, allowing PostgreSQL to locate matching rows via Bitmap Index Scan instead of scanning the full table. PostgreSQL's optimizer intelligently chose index scans for high-selectivity filters (0.9%) while keeping sequential scans for low-selectivity ranges (43%), confirming that indexes are most valuable when only a small fraction of rows match. The `auto_prescribe` stored procedure demonstrates database-level clinical decision support, mapping diagnoses to evidence-based medications and inserting prescriptions automatically.